Computación Concurrente, Paralela y Distribuida, Práctica 5

Paralelismo basado en pipelines

Versión: 3 de febrero de 2026

Contexto

En esta práctica **continuamos** con la ejecución de programas paralelos basados en la **paralelización con procesos** usando un **nuevo diseño** para el reparto de tareas y su ejecución. Mientras que en la P3 y P4 vimos como podemos usar la técnica de *Pooling* junto con un troceo y reparto de los datos a procesar, en esta práctica veremos como podemos hacer uso de una organización en forma de *pipeline*, también conocido en castellano como *segmentación* o *estructura de tuberías*.

Este diseño pone un especial hincapié en el papel de la *comunicación* entre los distintos procesos, dado que para que los **datos** puedan moverse por el *pipeline* es necesario **ingerirlos** o generarlos en el inicio de este, **procesarlos** en cada etapa, dirigirlos a la siguiente, y finalmente volcarlos, **consumirlos** o mostrarlos en el final del *pipeline*. Estas comunicaciones entre procesos son clave y en el aspecto técnico pueden llevarse a cabo de dos maneras, con *'pipes' (tuberías)*, y con *'queues' (colas)*. La diferencia entre ambas es que mientras las **tuberías** son **bloqueantes**, es decir, requieren que un dato escrito en la tubería sea leído para continuar con el programa, las **colas no** lo son, permitiendo introducir un dato en ella y seguir con la ejecución.

Finalmente, vamos a ver el efecto de aplicar un **segundo nivel de paralelismo** dentro de las etapas del pipeline, paralelizando su ejecución mediante la técnica ya vista de *Pooling*. Esto se hará con el objetivo principal de **reducir tiempos** en aquellas tareas que representen *cuellos de botella (bottleneck)*, es decir, que causan un 'atasco' en el *pipeline* que resulta en una **diferencia** de rendimiento **entre** unas **etapas** y otras.

Objetivos

En esta práctica usaremos dos códigos, uno que implementa un pipeline con '*pipes*', y otro que implementa el mismo pipeline con '*queues*', con los siguientes objetivos:

- Ver las **diferencias** entre comunicaciones **bloqueantes** y **no bloqueantes**
- Analizar el **efecto** de los **cuellos de botella** derivados de la diferente complejidad de las etapas
- Ver el **efecto** en los **tiempos** globales y por etapa de usar un **segundo nivel de paralelismo**

Índice

1 Pipeline con tuberías **3**
- 1.1 Explicación del código . . . 3
- 1.2 Construcción del pipeline . . . 5
- 1.3 Ejecución del pipeline . . . 6

2 Pipeline con colas **8**
- 2.1 Explicación del código . . . 8
- 2.2 Ejecución del pipeline . . . 10

Índice de Tareas

Tarea 1: Estudio del código basado en pipes 6

Tarea 2: Ejecución del código basado en pipes 7

Tarea 3: Estudio del código basado en queues 8

Tarea 4: Ejecución del código basado en queues, validación 10
Tarea 5: Ejecución del código basado en queues, experimentación 10
Tarea 6: Complejidad algorítmica de las funciones de las etapas 11

Antes de empezar: En esta práctica no es estrictamente necesario hacer uso de la MR dado que los códigos explotarán suficiente paralelismo como para apreciar una mejora incluso con pocos recursos. No obstante, se permite y recomienda ejecutar los códigos remotamente con los scripts proporcionados, lo que se puede hacer con los siguientes comandos:

```
$ bash execute_remotely.sh pipes
$ bash execute_remotely.sh queues
```

1. Pipeline con tuberías

1.1. Explicación del código

El pipeline implementado con tuberías se encuentra en el *código1_pipes.py* y realiza un procesamiento de una lista de números como entrada. En este fichero encontrarás una versión en secuencial del pipeline, donde a cada número de la lista se le aplican de forma consecutiva las funciones y se guarda su resultado (ver Figura 1). Observa esta implementación en las siguientes líneas:

Ejecución secuencia (codigo1_pipes.py)

```
def run_sequential():
    start = time.time()
    i = 0
    for item in data:
        item = filter_evens(item)
        if not item:
            continue
        item = is_prime(item)
        if not item:
            continue
        item = multiply_items(item)
        if not item:
            continue
        i += item
    end = time.time()
    print("[SEQ] finished after {0} ms".format(to_time_miliseconds(start, end)))
    print("[SEQ] End result is {0}".format(i))
```

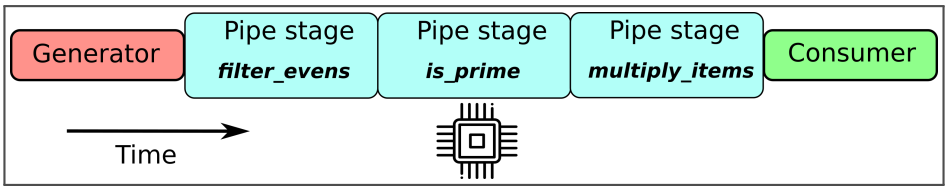

Figura 1: Ejecución secuencial del pipeline

También encontrarás la versión paralelizada en pipeline (ver Figura 2). Dejando de lado la primera etapa (Generator), que generará los números, y la última, que los consumirá (Consumer), sobre este input en forma de lista numérica se aplicarán 3 funciones, que representarán las etapas centrales del pipeline y serán el objeto de estudio. Estas funciones, en orden de ejecución, son:

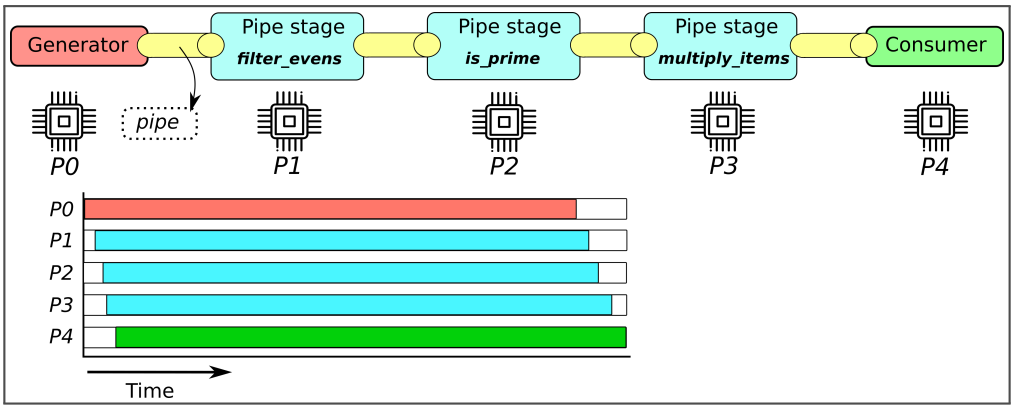

Figura 2: Ejecución con 'pipes' (comunicaciones bloqueantes)

- ***filter_evens***: filtrará los números pares
- ***is_prime***: filtrará todos los números que no sean primos
- ***multiply_items***: multiplicará cada número por si mismo

Puedes observar estas funciones en las siguientes líneas:

Las 3 etapas del pipeline (codigo1_pipes.py)

```
23 def multiply_items(item):
24     return item * item
25
26 def filter_evens(item):
27     if (item % 2) != 0:
28         return item
29
30 def is_prime(num):
31     flag = False
32     if num > 1:
33         for i in range(2, num):
34             if (num % i) == 0:
35                 flag = True
36                 break
37     if flag:
38         return None
39     else:
40         return num
```

Estas funciones serán llevadas a cabo por otra función que implementa una etapa de forma genérica, aplicando la función pasada como argumento. Puedes ver esta etapa en las siguientes líneas:

Implementación genérica (función abstraída) de etapa (codigo1_pipes.py)

```
53 def pipe_stage(pipe_in, pipe_out, funct):
54     _, input_pipe = pipe_in
55     output_pipe, _ = pipe_out
56     start = time.time()
57     try:
58         while True:
59             item = input_pipe.recv()
60             result = funct(item)
61             if result:
62                 output_pipe.send(result)
63     except EOFError:
64         end = time.time()
65         print("Stage '{0}' has finished after {1} ms".format(funct.__name__, to_time_miliseconds(start, end)))
66         output_pipe.close()
```

Finalmente, puedes a su vez observar las etapas que generan y consumen los números en las siguientes líneas:

Etapa del pipeline 'Generator' (codigo1_pipes.py)

```
43 def pipe_generator(pipe_out, data):
44     output_pipe, _ = pipe_out
45     start = time.time()
46     for item in data:
47         output_pipe.send(item)
48     end = time.time()
49     print("Stage 'generator' has finished after {0} ms".format(to_time_miliseconds(start, end)))
50     output_pipe.close()
```

Etapa del pipeline 'Consumer' (codigo1_pipes.py)

```
69 def pipe_consumer(pipe_in):
70     i = 0
71     try:
72         while True:
73             i += pipe_in[1].recv()
74     except EOFError:
75         print("[PIPELINE] End result is {0}".format(i))
```

1.2. Construcción del pipeline

Centrándonos ya en la versión paralelizada que usa un pipeline con varias etapas, podemos comentar de forma más detallada cómo se construye este pipeline observando la función 'run_pipeline', donde se llevan a cabo varias tareas. Cogiendo como referencia un subconjunto de líneas de esta función, podemos ver algunas:

Construcción del pipeline (codigo1_pipes.py)

```
120     pipe3 = multiprocessing.Pipe(duplex=True)
121     p3 = entity_type(target=pipe_stage, args=(pipe2, pipe3, is_prime,))
122     pipe_list.append(pipe3)
123     process_list.append(p3)
124
125     pipe4 = multiprocessing.Pipe(duplex=True)
126     p4 = entity_type(target=pipe_stage, args=(pipe3, pipe4, multiply_items,))
127     pipe_list.append(pipe4)
128     process_list.append(p4)
129
130     # Consumer
131     p5 = entity_type(target=pipe_consumer, args=(pipe4,))
132
133     start = time.time()
134
135     if implementation == "process":
136         for pipe, p in zip(pipe_list, process_list):
137             p.start()
138             pipe[0].close()
```

- Creación de los 'pipes' para comunicar (líneas 120 y 125)
- Creación de los procesos/hilos para implementar las etapas (líneas 121, 126 y 131)
- Mapeo ordenado de funciones a procesos/hilos (etapa 'is_prime' en la línea 121, etapa 'multiply_items' en la 126 y etapa de consumición final en la 131, ver tercer argumento de tupla o parámetro 'target')
- Conexión de 'pipes' (líneas 121, 126 y 131, cuando se especifican los pipes a usar)
- Arranque de los procesos/hilos (líneas 136 y 137)

Tarea 1 – Estudio del código basado en pipes

Analiza el resto de la función *run_pipeline* para comprender más en detalle la construcción del pipeline, especialmente el orden de las funciones a ejecutar en las tareas centrales, así como la manera en la que se realiza la 'conexión' de los 'pipes'.

Python Pipes y el Sistema Operativo: Las tuberías de Python, o 'pipes', que estamos creando y usando en este código usan realmente los propios mecanismos que proporciona el Sistema Operativo para llevar a cabo comunicaciones entre hilos y procesos (e.g., tuberías Linux). Estas tuberías se comentaron en el Tema 1, cuando se explicó las formas de comunicación de los procesos con su entorno. No obstante, la gestión de la librería de Python de estas tuberías simplificará su manejo.

1.3. Ejecución del pipeline

Antes de ejecutar este código, hay que recordar que paralelizar no siempre implica aprovechar los recursos y obtener una ganancia equivalente, o ni siquiera cercana. En este caso hemos aplicado una técnica de paralelización que asigna funciones a procesos y mueve los datos a través de ellos, pero esto no significa que necesariamente todos los procesos estén aplicando la función todo el tiempo. A diferencia de prácticas anteriores, donde hacíamos 'cálculo puro' sobre números, en este caso el pipeline y su rendimiento se verán muy influenciados por factores externos, principalmente:

- **Los datos a procesar** → para este pipeline no es lo mismo procesar una lista desordenada de números aleatorios (trabajan todas las etapas), que una con solo números pares (solo trabaja la primera etapa), o una sin números primos (solo trabajan primera y segunda etapa).

- **El orden de las funciones** → teniendo en cuenta que SIEMPRE deben cumplir con el algoritmo a implementar, en este caso podríamos intercambiar la primera y la segunda etapa. Esto implicaría que se comprobarían y filtrarían todos los números en busca de los primos, y luego se eliminarían los pares. Este orden inutilizaría la segunda etapa (los pares ya se habrían eliminado al no ser primos) y además aumentaría la carga de la primera (habría muchos más números a comprobar). Quizás con este orden, si comparamos una versión secuencial y una paralela, veríamos una mejora sustancial, pero en el fondo la forma de implementar el algoritmo sería ineficiente y su optimización es preferible a la ganancia con el paralelismo.
- **La complejidad de las funciones** → no todas las funciones tienen el mismo coste. Algunas tienen un coste computacional mínimo (filter_evens), mientras que otras tienen un coste alto (is_prime).

También es interesante comentar que las comunicaciones que usan la librería de 'pipes' presente en este código serán de tipo *bloqueante*. Esto quiere decir que estas comunicaciones obligarán a uno de los dos extremos a esperar. Si el emisor quiere enviar un dato, tendrá que esperar a que el receptor lo reciba, y de igual forma, si el receptor quiere un nuevo dato del pipe, tendrá que esperar a que el emisor lo mande.

De igual forma, es destacable mencionar que, siendo un pipeline una estructura que 'encadena' etapas y procesamiento con unas tareas principio y final, se puede ver así como un grafo acíclico dirigido (DAG). Esto nos permite aplicarle el concepto de *camino crítico*, que representa la ruta de procesamiento más larga o costosa. Aunque en este caso el pipeline solo tiene un camino y por ello este ya sería el camino crítico, hay escenarios donde un flujo de datos puede recorrer varios caminos con distintas características. En estos casos sería preferible aplicar el paralelismo sobre dicha ruta a fin de ver una ganancia.

Finalmente, también podemos aplicar otro concepto relacionado con los pipelines que sí que nos será útil, el de *cuello de botella*, que define aquella tarea que consume la mayor cantidad de tiempo. Por un lado, este concepto es especialmente importante en escenarios bloqueantes, dado que determinará el coste temporal global de procesar los elementos en el *pipeline*. En estos escenarios, la tarea que represente el cuello de botella 'bloqueará' al resto y las obligará a esperar por ella, provocando que sus tiempos de ejecución sean iguales o mayores. Por otro lado, en escenarios no bloqueantes, el *cuello de botella* nos indicará donde está la etapa a optimizar dentro del pipeline, teniendo en cuenta que mejorar la ejecución de otra etapa reducirá su tiempo propio, pero no el del pipeline en global si se producen esperas.

Tarea 2 – Ejecución del código basado en pipes

Ejecuta ahora el *'código1_pipes'*:

- **¿Es el resultado final el mismo usando pipelines que ejecutando de forma secuencial?** (test de validación)
- Con un rango de datos de 10 a 2e5, **¿Tarda lo mismo la versión secuencial que la paralela con hilos? ¿Y con procesos hay ganancia?**
- **¿Si ambas versiones paralelas tardan lo mismo o incluso más, a qué crees que se debe?**
- **¿Tardan lo mismo todas las funciones, o algunas son más rápidas?**

2. Pipeline con colas

2.1. Explicación del código

En el *código2* encontrarás otra implementación de un pipeline, pero esta vez usando colas en vez de tuberías. Las colas son estructuras de comunicación que, a diferencia de las tuberías, están pensadas para ser usadas por varios procesos o hilos y que además pueden almacenar elementos a fin de no bloquear, es decir, son *no bloqueantes*. Puedes ver el uso de estas colas en la siguiente función, que en este caso es la encargada de construir cualquier pipeline de forma genérica a partir de una lista de funciones:

Función que construye un pipeline basad en colas (codigo2_queues.py)

```
def construct_pipeline(queue_stage_type):
    manager = multiprocessing.Manager()
    measurements = manager.dict()

    process_list, queues_list = list(), list()
    q = multiprocessing.Queue(maxsize=QUEUE_SIZE)
    process_list.append(multiprocessing.Process(target=queue_generator, args=(q, data, measurements,)))
    queues_list.append(q)
    for fun in PIPELINE_FUNCS:
        q = multiprocessing.Queue(maxsize=QUEUE_SIZE)
        process_list.append(
            multiprocessing.Process(target=queue_stage_type, args=(queues_list[-1], q, fun, measurements,)))
        queues_list.append(q)
    process_list.append(multiprocessing.Process(target=queue_consumer, args=(queues_list[-1], measurements,)))
    return process_list, measurements
```

El resto del código implementa la solución en secuencial y con un pipeline paralelizado, pero con un enfoque más genérico y flexible que en el *código1*, permitiendo construir un pipeline para cualquier combinación de funciones. Esto a su vez se aprovecha para estudiar un pipeline distinto con otras funciones de mayor complejidad algorítmica en general.

Además, en este código se ha implementado un segundo nivel de paralelismo que permite, si se desea, paralelizar con *Pooling* las etapas. Puedes ver visualmente la construcción de este pipeline, en comparación con el que hace uso de tuberías, en la Figura 3. De igual forma, en la Figura 4 puedes ver gráficamente el segundo nivel de paralelismo.

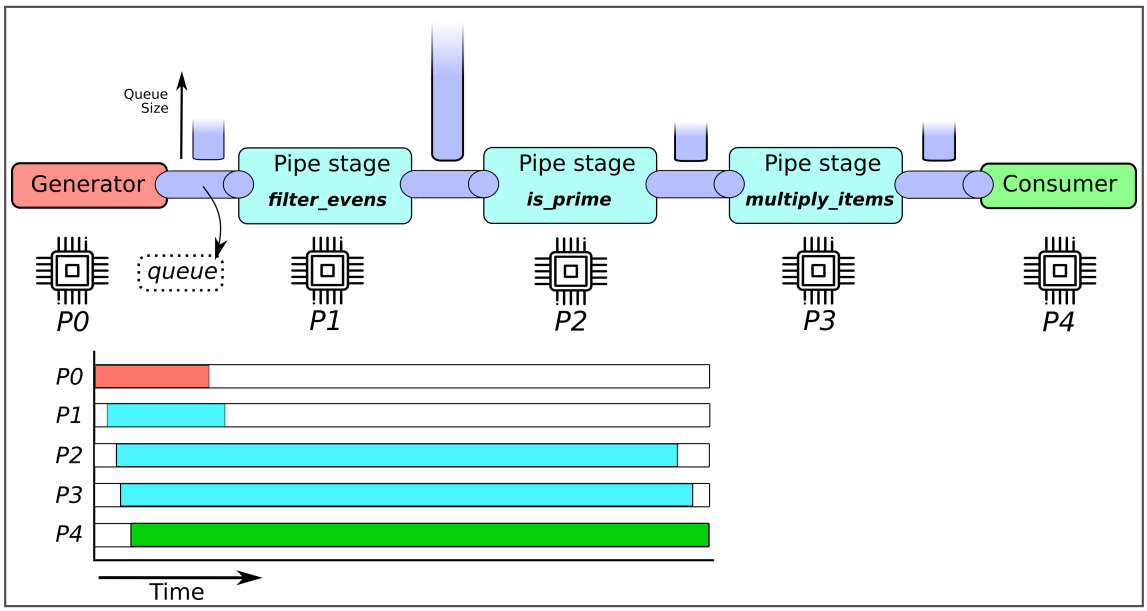

Figura 3: Ejecución con 'queues' (comunicaciones NO bloqueantes) – Paralelismo simple

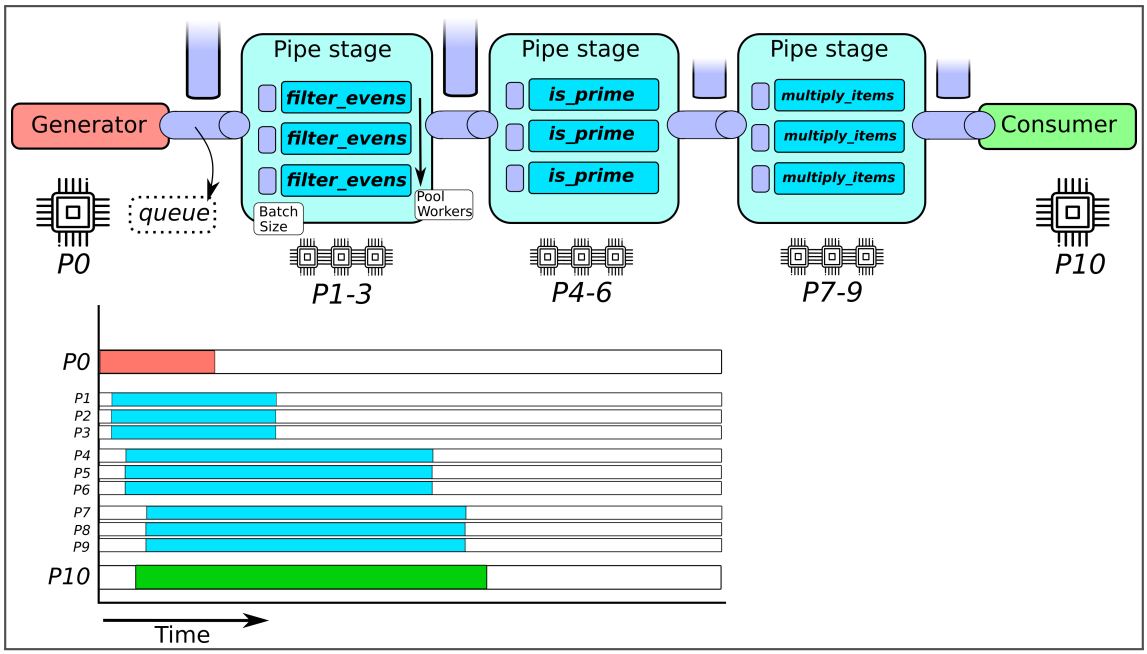

Figura 4: Ejecución con 'queues' (comunicaciones NO bloqueantes) – Paralelismo múltiple (con *Pooling*)

Tarea 3 – Estudio del código basado en queues

Analiza el resto del *'código2_queues'*. Estudia en particular la función *'queue_stage_parallel'*, que implementa la ejecución de una etapa con paralelsimo basado en Pooling.

Es de interés destacar lo muy configurable que es este código en lo que respecta al grado de paralelismo que se puede aplicar, el cual podemos regular con varias variables:

- Variable **QUEUE_SIZE** ⇒ tamaño de las colas a usar entre etapas:
  - Colas pequeñas → más fácil producir un bloqueo al llenarse la cola pronto
  - Colas grandes → potencial gasto de memoria si se llenan mucho, en un caso extremo (generación de datos infinita) se podría agotar la memoria del sistema
- Variable **POOL_WORKERS** ⇒ tamaño del pool de trabajadores para la ejecución en paralelo dentro de las etapas:
  - Pocos trabajadores → estos podrían soportar mucha carga o no dar abasto, provocando un cuello de botella
  - Muchos trabajadores → soportarán menos carga y no causarán cuello de botella, pero podría haber un overhead de gestión o incluso trabajadores ociosos
- Variable **BATCH_SIZE** ⇒ tamaño de las tareas del pool para el paralelismo en las etapas (segundo nivel de paralelismo), en esencia el número de números en cada tarea:
  - Tamaños muy pequeños → implicarán la generación y reparto de muchas tareas, con su gasto de gestión, pero asegura que todos los workers estarán haciendo algo en todo momento.
  - Tamaños grandes → pueden obligar a la etapa a esperar a tener suficientes datos para mandar una tarea, haciendo que los trabajadores estén ociosos, pero ahorra trabajo de gestión.
- Variable **PIPELINE_FUNCS** → funciones a ejecutar en el pipeline (sin contar generador inicial y consumidor final)

Además, hay 4 maneras distintas de ejecutar el pipeline aprovechando la combinación de técnicas de paralelismo:

- *sequential* → ejecución secuencial de las funciones sobre la lista de números
- *parallel* → ejecución en paralelo de las etapas, en secuencial dentro de la etapa
- *parallel_multi_threads* → ejecución en paralelo de las etapas y dentro de la etapa, con hilos
- *parallel_multi_process* → ejecución en paralelo de las etapas y dentro de la etapa, con procesos

***Process oversubscription*:** Ten en cuenta que en estos códigos la variable **POOL_WORKERS** indica el número de workers que se crearán para paralelizar **cada una de las etapas intermedias** del pipeline, por lo que si el pipeline tiene 3 etapas (sin contar generación y consumición), y **POOL_WORKERS** vale 4, las etapas intermedias usarán un total de 12 procesos.

Este *oversubscription*, o sobrecarga de procesos al no disponer probablemente de tantos cores en el sistema, **NO es pernicioso en este contexto**. Lo aceptamos porque ya sabemos de antemano que muchas de las etapas no explotarán este paralelismo debido a que sus funciones son simples (incluso les puede penalizar ligeramente), y que el tiempo que no aprovechen unos procesos de una etapa puede ser aprovechado por otros de otra etapa.

2.2. Ejecución del pipeline

Tarea 4 – Ejecución del código basado en queues, validación

Ejecuta ahora el *'código2_queues'*, deberías encontrar descomentadas secciones para ver la validación del pipeline usando los resultados de la ejecución secuencial para comparar.

Tarea 5 – Ejecución del código basado en queues, experimentación

Descomenta ahora y ejecuta el resto de secciones y experimentos del *'código2_queues'*, a fin de ver el papel de las variables anteriormente explicadas a la hora del modular el paralelismo y el rendimiento obtenido. Puedes ir descomentando y ejecutando secciones individuales (ejecutar todas puede llevar un tiempo considerable, varios minutos):

- Salvo en la ejecución secuencial, **¿observas diferentes tiempos de ejecución para las distintas etapas? ¿A qué se debe esta diferencia con el código1?**
- **¿Qué etapa está haciendo de cuello de botella?, o dicho de otra manera, ¿qué etapa determina el tiempo de ejecución del pipeline?**
- **¿Observas una ganancia entre ejecutar el pipeline en secuencial o en paralelo simple, es decir sin paralelización en la ejecución interna de las etapas? ¿A qué se debe esto?**
- **¿Observas una ganancia entre usar el paralelismo dentro de las etapas para todas las funciones, o hay funciones que no se benefician? ¿A qué se puede deber esto?**
- Si comparas las diferentes configuraciones de ejecución en paralelo de los pipelines, **¿observas alguna diferencia entre la ejecución interna en secuencial (parallel) o en paralelo de las etapas (parallel_multi_*)? ¿Obtiene el mismo resultado usar hilos y procesos?**
- **¿Qué efecto tiene cada una de las siguientes variables?**
  - **POOL_WORKERS**
  - **QUEUE_SIZE**
  - **BATCH_SIZE**

  (Puedes probar a usar otros valores a los presentes en el código entregado)

Timeout de las ejecuciones en la MR: Si ejecutas el código *'código2_queues'* en la MR es recomendable que lo hagas comentando y descomentando secciones de este, dado que la ejecución al completo puede sobrepasar el timeout.

Algoritmia y complejidad

Tarea 6 – Complejidad algorítmica de las funciones de las etapas

Esta es una tarea optativa/preparatoria para P6

Como se ha mencionado anteriormente, la complejidad de las funciones puede ser crucial a la hora de entender cómo se puede mejorar el rendimiento de un programa mediante paralelismo, o si directamente es posible o merece la pena intentarlo. Para poder realizar este análisis es necesario recurrir al análisis de algoritmos y más específicamente, al análisis de la complejidad temporal de los algoritmos (link a wikipedia).

Analiza las funciones del *código2* e intenta ordenarlas por su complejidad, es decir, lee su código y compáralas unas con otras para ver cuáles están en el mismo rango de complejidad, y cuáles son más complejas.

A continuación se listan las funciones y su notación 'O grande', comprueba si estás de acuerdo:

- filter_divisible_by → O(24) = O(1)
- is_prime → O(sqrt(n))
- checksum → O(n)
- filter_evens → O(1)
- is_emirp → O(n)

Copyright & Disclaimer


```
# Copyright © 2025 by Jonatan Enes (jonatan.enes@udc.es)
# Computer Engineering department, Universidade da Coruña, Spain.
#
# This file is part of several courses on parallel processing from Universidade da Coruña,
# and it can only be used and/or modified by the author, the students or any
# explicitly authorized person by the author, inside the context of the subject.
# Redistribution to a third-party without previous authorization is strictly forbidden.
# No commercial use is allowed.
#
# This file has only academic purposes and should not be used for any real-world
# scenario, the author holds no accountability for its use or misuse.
```